*AI Engineering and Claude Architect · Part I, Foundations*

# Working With Claude, Lesson 3

## Structured Output and Validation Loops

The JSON parses perfectly. The schema passes. The line items add up to $150, and the total says $200. Valid JSON tells you the format is right, and says nothing about whether the content is true. This lesson builds the layers that catch the difference.

| | |
|---|---|
| Module | AI03 of 13 |
| Lesson | 3 of 6 |
| Prerequisites | AI03 L1–L2, AI02 L6 |
| Exam relevance | **Direct and heavy.** TS 4.3 and TS 4.4, Scenario 6, and `tool_choice` in TS 2.3. This lesson carries an important difference between the exam and current models |
| Reading time | ~50 minutes |

# Contents

1. Structured Output and Validation Loops

*Lesson 3 of 6 · Working With Claude*

# Structured Output and Validation Loops

### *Key idea*

**Schema enforcement eliminates syntax errors, not wrong answers.** So reliable extraction needs layers: a schema for structure, nullable fields so the model can honestly report absence, business validation in code for meaning, and retries that include the specific error. **A retry can fix a mistake. It can't conjure up information that isn't in the document, so know when to stop and call a human.**

> *Before: fabrication and hallucination (AI02 L6), prompt structure (AI03 L2). Teaches: the routes to structured output, `tool_choice`, schema design, semantic validation, retry with error feedback, its limits, and detected-pattern feedback. Used by: AI06 L3, AI08 L5, AI10 L3, AI13 L5.*

## The hook: three invoices

Your accounts-payable pipeline extracts invoices into your ERP. It uses a strict schema, and every response is valid JSON. Then three problems arrive in one week.

| Invoice | Output | Problem | Would a schema catch it? |
|---|---|---|---|
| A | `"purchase_order": "PO-48213"` | The invoice has no PO. The model invented one, because the field is required | **No** |
| B | `"stated_total": 200.00`, with items summing to 150.00 | Either a misread, or the source disagrees with itself | **No** |
| C | `"currency": "dollars"` | Should be `USD` | Only if you constrain it |

*Key idea*

**All three are schema-valid, and all three are wrong.** TS 4.3 says exactly this: strict JSON schemas eliminate syntax errors, but don't prevent semantic errors, such as line items that don't sum to the total, or values in the wrong fields.

And invoice A is worse than the others, because **your own schema caused it**. A required field is a demand. If the document doesn't have the value, the model is under pressure to fill it anyway.

## Intuition: a form and an auditor

A paper form can force a date into a date box. It can't stop someone writing the wrong date. So real organizations have a form **and** an auditor. The auditor checks that totals add up, sends a form back with a specific note, "line 3 doesn't match the receipt", and knows when to stop: "the receipt is missing, and asking again won't produce it."

| Layer | Analogy | Catches |
|---|---|---|
| JSON schema | The form's boxes | Wrong shape and syntax |
| Nullable fields | A "not provided" checkbox | Pressure to invent |
| Code validation | The auditor | Wrong meaning, as far as rules can tell |
| Retry with feedback | Sending it back with a note | Fixable mistakes |
| Human review | The senior accountant | Everything else |

## Getting structured output: the options

| Approach | How | Guarantee | Status |
|---|---|---|---|
| Ask for JSON in the prompt | "Respond with JSON" | None. May add prose or break syntax | Weakest |
| Assistant prefill | Start the reply with `{` | None | **Returns 400 on current models** |
| **Tool use with a schema** | Define a tool, and read its `tool_use.input` | Model produces tool arguments. Exam's named approach | **Tested on the exam** |
| **Strict tool use** | `strict: true` on the tool | Tool inputs validated against the schema | Current API |
| **Structured outputs** | `output_config.format`, or `.outputConfig(MyRecord.class)` in Java | Response text is valid JSON matching the schema | Current API |

## The exam's answer, and what to build today

> *Know both. The exam and current models differ here.*
>
> **On the exam:** tool use with JSON schemas is "the most reliable approach for guaranteed schema-compliant structured output" (TS 4.3), with `tool_choice` controlling whether a tool must be called. **Answer exam questions this way.**
>
> **In production today:** on Claude Opus 5.5, Sonnet 5.5, Fable 5.1 and Mythos 5.1, **forced `tool_choice` (`any` or a named tool) returns a 400.** Anthropic's reference recommends `output_config.format` when the forced call only existed to get JSON. Alternatively, use `auto`, an explicit instruction naming the tool, and `strict: true`, then check that a call was actually made.

## `tool_choice` explained

| Value | Behavior | Exam use case | Current models |
|---|---|---|---|
| `{"type": "auto"}` | Claude decides, and may return text | Default. Doesn't guarantee a tool call | Works everywhere |
| `{"type": "any"}` | Must call a tool, of its choice | Several extraction schemas, with unknown document type | 400 on Opus 5.5, Sonnet 5.5, Fable 5.1 |
| `{"type": "tool", "name": "x"}` | Must call that tool | Force `extract_metadata` to run before enrichment | 400 on those models. Works on models such as Opus 5 and Opus 4.8 |
| `{"type": "none"}` | Can't call tools | Not in the guide | Works everywhere |

Memorize the exam column. It's a reliable source of questions, and the guide's appendix lists all of the first three.

## Both routes in Java

The current route. The SDK derives the schema from a record, and returns a typed object. It compiles against `anthropic-java` 3.2.0, and hasn't been run, since no API key is available.

```
public record ExtractedOrder(String orderId, String deliveryDate, String notes) {
}

public Optional<ExtractedOrder> extract(String document) {
    StructuredMessageCreateParams<ExtractedOrder> params = MessageCreateParams.builder()
            .model(model)
            .maxTokens(4_096L)
            .outputConfig(ExtractedOrder.class)
            .addUserMessage("<document>" + document + "</document>\nExtract the order.")
            .build();
    return client.messages().create(params).content().stream()
            .flatMap(block -> block.text().stream())
            .map(typed -> typed.text())
            .findFirst();
}
```

The exam's route, using a forced tool on a model that supports it:

```
Tool tool = Tool.builder()
        .name("record_order")
        .description("Record the order fields found in the document.")
        .strict(true)
        .inputSchema(Tool.InputSchema.builder()
                .properties(Tool.InputSchema.Properties.builder()
                        .putAdditionalProperty("order_id",
                                JsonValue.from(Map.of("type", "string")))
                        .putAdditionalProperty("delivery_date", JsonValue.from(Map.of(
                                "type", List.of("string", "null"))))
                        .build())
                .required(List.of("order_id", "delivery_date"))
                .putAdditionalProperty("additionalProperties", JsonValue.from(false))
                .build())
        .build();
```

Read `delivery_date` carefully. It's in `required`, and its type allows `null`. So the key must appear, while its value can honestly say "not found". That's the nullable pattern, and it's compatible with strict mode.

## Strict schemas have limits

From Anthropic's structured outputs documentation:

| Supported | Not supported, returning a 400 |
|---|---|
| Basic types, `enum`, `const` | Recursive schemas |
| `anyOf`, and `allOf` with limits | Numeric limits: `minimum`, `maximum` |
| `$ref` within the schema | String limits: `minLength`, `maxLength` |
| Formats such as `date`, `email`, `uuid` | Array limits beyond `minItems` of 0 or 1 |
| `additionalProperties: false` (required on objects) | `additionalProperties` set to anything else |

So **"amount must be positive" can't be enforced by the schema here.** It needs code. Other details from that page:

| Detail | Consequence |
|---|---|
| First use of a schema compiles a grammar | Extra latency once. Cached for 24 hours from last use |
| Required properties appear first in output | Don't depend on field order |
| `stop_reason: "refusal"` | Output may not match the schema |
| `stop_reason: "max_tokens"` | Output may be incomplete. Retry with a higher limit |
| Limits: 20 strict tools, 24 optional parameters, 16 union-typed parameters per request | Keep schemas simple |

So **always check `stop_reason`, even with structured outputs.** AI03 L1's rule still applies.

# Designing the schema

## Required, optional or nullable?

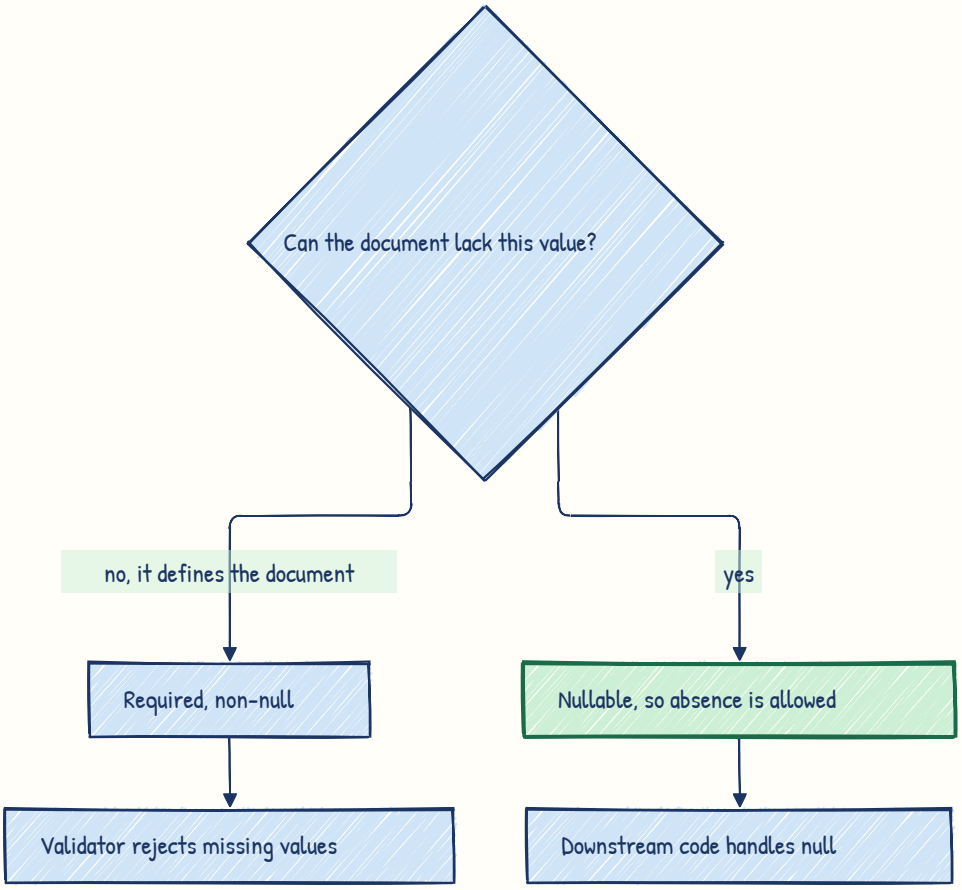



*Figure 1. Choosing field semantics. The key question is whether the source can legitimately lack this value.*

TS 4.3 is explicit: design fields as optional or nullable when source documents may not contain the information, preventing the model from fabricating values to satisfy required fields.

## Enums need an escape hatch

A closed enum has the same problem as a required field: it forces a choice.

| Pattern | Example | Prevents |
|---|---|---|
| `"other"` + detail | `category: "other"`, `category_detail: "customs duty"` | Forcing new categories into the wrong bucket |
| `"unclear"` | `category: "unclear"` | Guessing on ambiguous documents |

```
public enum Category { GOODS, SERVICES, OTHER, UNCLEAR }

public record Invoice(
        String invoiceNumber,
        String purchaseOrder,
        String currency,
        BigDecimal statedTotal,
        List<LineItem> items,
        Category category,
        String categoryDetail) {
}
```

And `"other"` also gives you **data**: reviewing `category_detail` values shows you which categories to add next.

### Normalization rules

Schemas don't handle inconsistent source formatting. TS 4.3 says to **include format normalization rules in prompts alongside strict output schemas**: "Dates as YYYY-MM-DD. Amounts as decimals without currency symbols. Currency as ISO 4217 codes."

### Add fields that let the model check itself

| Field | Purpose |
|---|---|
| `stated_total` and `calculated_total` | Extract both, then flag any difference |
| `conflict_detected` | A boolean for source data that disagrees with itself |
| `field_confidence` | Used only after calibration (TS 5.5, AI10 L3) |
| `source_quote` | Lets a reviewer verify the value quickly |
| `detected_pattern` | For review feedback loops, explained below |

## Validation: the semantic layer

Here's the auditor, from `code/ai03-claude/.../Extraction.java`. Each problem has a **kind**, because that decides what happens next.

```java
public enum Kind { FORMAT, SEMANTIC, ABSENT }

public record Problem(Kind kind, String message) {
    boolean retryable() {
        return kind != Kind.ABSENT;
    }
}
```

```java
static List<Problem> validate(Invoice invoice) {
    List<Problem> problems = new ArrayList<>();
    if (invoice.currency() == null || !invoice.currency().matches("[A-Z]{3}")) {
        problems.add(new Problem(Kind.FORMAT,
                "currency must be a 3-letter ISO code, got: " + invoice.currency()));
    }
    if (invoice.statedTotal() == null) {
        problems.add(new Problem(Kind.ABSENT, "no stated total in the document"));
    } else {
        BigDecimal sum = invoice.items().stream()
                .map(LineItem::amount)
                .reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(invoice.statedTotal()) != 0) {
            problems.add(new Problem(Kind.SEMANTIC, "line items sum to " + sum
                    + " but the stated total is " + invoice.statedTotal()));
        }
    }
    return problems;
}
```

| Error class | Example | Caught by | Retry helps? |
|---|---|---|---|
| **Syntax** | Missing brace | Tool use or structured outputs prevent it | Not needed |
| **Format** | `"dollars"` for currency | Code, or an enum | Usually |
| **Semantic** | Total doesn't match items | Code only | Sometimes |
| **Absent** | Value isn't in the document | Null plus a rule | **No** |

The appendix mentions **Pydantic** as the validation tool. It's Python's standard choice, and it's what you'd see in exam text. In Java, the equivalents are records plus Jakarta Bean Validation for field rules, and plain code for cross-field rules like this sum.

## The retry loop

**Retry with error feedback** means sending a follow-up containing three things: the original document, the failed extraction, and the specific validation error (TS 4.4).

```
static String feedback(Invoice failed, List<Problem> problems) {
    StringBuilder sb = new StringBuilder("Your previous extraction was:\n")
            .append(failed).append("\nIt failed these checks:\n");
    problems.forEach(p -> sb.append("- ").append(p.message()).append('\n'));
    return sb.append("Re-read the document and correct only these fields. If a value ")
            .append("isn't in the document, use null.").toString();
}
```

```
public static Result run(Extractor extractor, String document, int maxAttempts) {
    String feedback = null;
    Invoice invoice = null;
    List<Problem> problems = List.of();
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        invoice = extractor.extract(document, feedback);
        problems = validate(invoice);
        if (problems.isEmpty()) {
            return new Result.Accepted(invoice, attempt);
        }
        // Absent information can't be found by asking again.
        if (problems.stream().noneMatch(Problem::retryable)) {
            return new Result.NeedsReview(invoice, problems, attempt);
        }
        feedback = feedback(invoice, problems);
    }
    return new Result.NeedsReview(invoice, problems, maxAttempts);
}
```

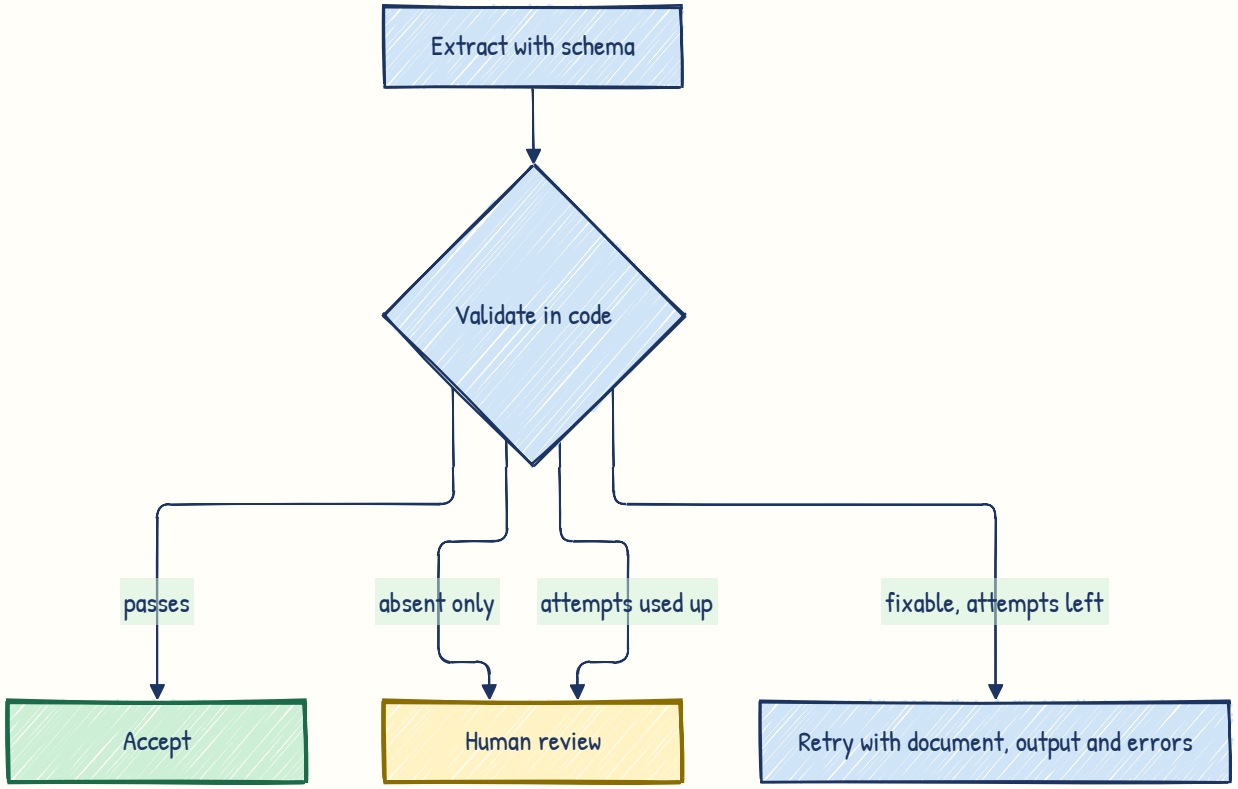



*Figure 2. The validation loop. Absent data goes straight to a person. Fixable problems get specific feedback, and persistent ones end up with a person as well.*

Real output, using scripted model answers so that the loop's logic runs with no API key. The answers are canned, so this tests the loop, not Claude.

```
1. Format error, fixable
  feedback sent: [- currency must be a 3-letter ISO code, got: dollars]
   result: ACCEPTED after 2 attempt(s)
2. Purchase order absent, and the field is nullable
   result: ACCEPTED after 1 attempt(s)
3. Total missing from the source
   result: HUMAN REVIEW after 1 attempt(s): [ABSENT]
4. Source really does contradict itself
  feedback sent: [- line items sum to 150.00 but the stated total is 200.00]
  feedback sent: [- line items sum to 150.00 but the stated total is 200.00]
   result: HUMAN REVIEW after 3 attempt(s): [SEMANTIC]
```

| Case | What happened | The lesson |
|---|---|---|
| 1 | Fixed by one specific note | Format errors respond to retries |
| 2 | Null accepted first time | Nullable fields prevent invention, and avoid a false failure |
| 3 | No retry at all | **Retrying absent data wastes money, and invites fabrication** |
| 4 | Two wasted retries | A real conflict in the source won't resolve. Flag it instead |

**Improve case 4 in production:** add `conflict_detected`. If the model can see that the source itself disagrees, it can report that, and you can skip the retries. That's what the guide's `calculated_total` and `conflict_detected` pattern is for.

### *The limits of retry (TS 4.4)*

Retries succeed on **format mismatches and structural errors**. They fail when **the required information is absent**, for instance when it's only in an external document you didn't provide. Exam questions often describe exactly this: repeated retries on a document that doesn't have the data. Don't retry. Route it to a person, or fetch the missing document.

## Feedback loops: learning from dismissals

Here's a pattern that's in the guide, and that's easy to miss: add a `detected_pattern` field to each finding, recording which code construct triggered it.

```
{
  "location": "OrderService.java:88",
  "issue": "Possible null dereference",
  "severity": "medium",
  "detected_pattern": "optional_get_without_check"
}
```

When developers dismiss findings, you can then group by pattern:

| detected_pattern | Reported | Dismissed | Dismissal rate | Action |
|---|---|---|---|---|
| `optional_get_without_check` | 120 | 12 | 10% | Keep |
| `magic_number` | 200 | 170 | 85% | Disable, or tighten the criteria |

These figures are illustrative. Without the field, you only know that "developers dismiss a lot". With it, you can see which **prompt change** would help. The structured output is feeding your improvement process.

## How Anthropic approaches it

| Problem | What Anthropic provides |
|---|---|
| Syntax errors in JSON | Structured outputs and strict tool use, using compiled grammars |
| Prefill was a workaround | Removed. Structured outputs replace it |
| Hand-writing schemas | SDK helpers derive them from types, in Java from records |
| Model-specific behavior | Forced `tool_choice` removed on some models, with documented alternatives |
| Source verification | Citations that point to document passages. Note that the reference says citations are incompatible with `output_config.format` |
| Knowing what was fabricated | Nothing automatic, so your validation remains essential |

## The production architecture

**This is a recommended reference design, not a specific company's system.** Scenario 6 on the exam describes this system.

**Baseline: extract and store.** The schema passes, so everything goes into the ERP. Fabricated POs and mismatched totals enter the books.

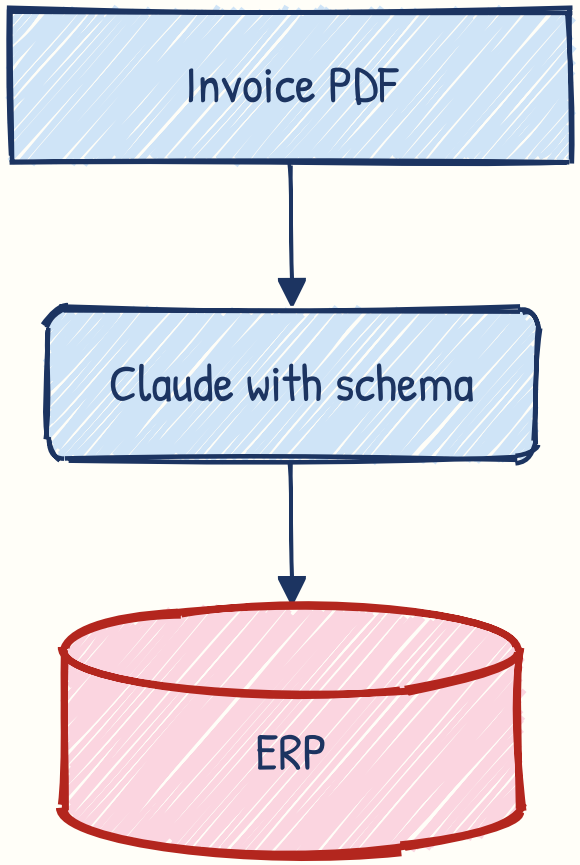



*Figure 3. Baseline. Schema-valid output goes directly into the system of record, including any wrong values.*

**Improved: validate and retry.** Semantic checks catch much of it. New problem: **failed documents retry forever or vanish, and nobody measures whether "accepted" really means correct.**

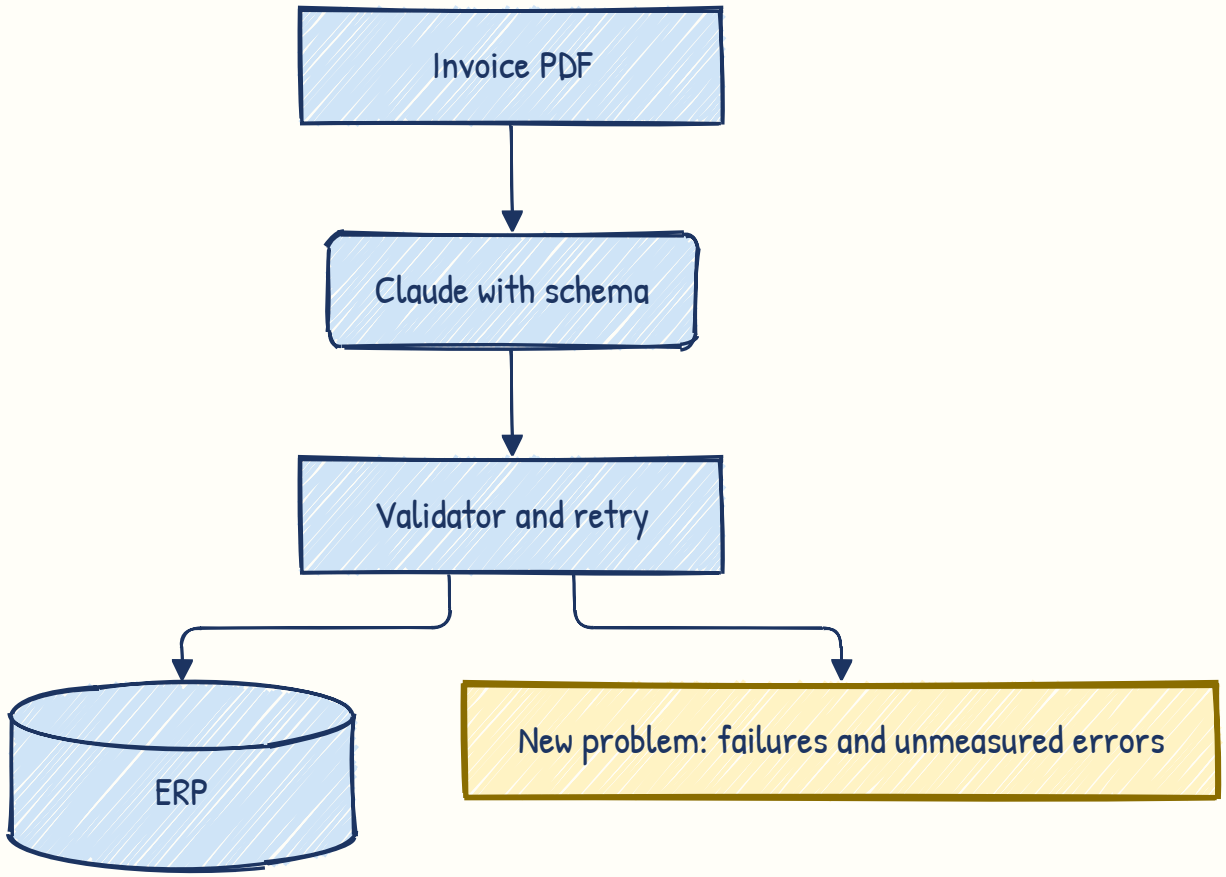



*Figure 4. Improved. Validation catches rule violations, but there's no destination for what fails, and no measurement of what passes.*

Production: every document has a defined path, and quality is sampled.

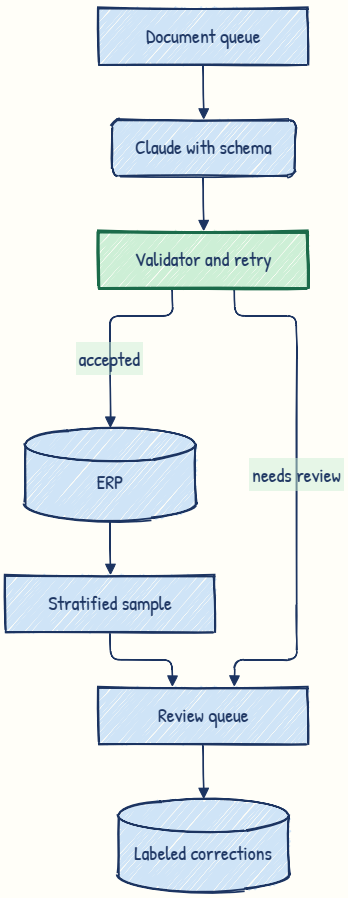



*Figure 5. Production. Validated output reaches the ERP, failures reach reviewers, and a stratified sample of accepted output keeps measuring real accuracy. Corrections become eval cases.*

| Component | Why it exists | How it communicates |
|---|---|---|
| Document queue | Absorbs volume, and allows batch processing | Kafka or a work queue |
| Validator | Applies business rules the schema can't | In-process |
| Review queue | A destination for anything uncertain | Work queue, with source quotes attached |
| Stratified sample | "Accepted" doesn't mean "correct" (TS 5.5) | Scheduled job by document type |
| Labeled corrections | Become eval data and few-shot candidates | Database (AI10 L3) |

### Failures

| Failure | Containment |
|---|---|
| Schema compile adds latency on first use | Warm schemas after deployment |
| `max_tokens` on a long document | Check `stop_reason`, then raise the limit or chunk it |
| Refusal on sensitive content | Route to a person |
| Retry storm during an outage | Cap attempts, back off, and use a dead-letter queue |
| Prompt change lowers accuracy on one document type | Evals by segment, before release |
| Model invents plausible values | Nullable fields, `source_quote` checks, sampling |

## Trade-offs

| Decision | Gain | Cost |
|---|---|---|
| Structured outputs | Valid JSON, and simple code | Schema limits, and compile latency once |
| Tool use | Fits agent loops, and it's the exam's model | Must read `tool_use` blocks |
| Nullable fields | Honest absence | Downstream must handle null |
| Many required fields | Simple consumers | Encourages invention |
| More retries | Fixes more errors | Cost and latency, with diminishing returns |
| More validation rules | Catches more | Must be maintained, and can reject valid input |

**When not to retry at all:** if the problem kind is "absent", or the same error has returned after one attempt with specific feedback.

### Common mistakes

1. **Thinking valid JSON means correct data.** It only means valid JSON.
2. **Marking everything required.** It pushes the model to invent values.
3. **Closed enums with no escape.** Add `other` and `unclear`.
4. **Retrying with "try again".** Send the document, the output and the specific error.
5. **Retrying absent information.** A guaranteed waste.
6. **Skipping `stop_reason` with structured output.** Refusals and truncation can break the schema.
7. **Sending forced `tool_choice` to Opus 5.5.** You'll get a 400 today, though the exam still tests the concept.
8. **Measuring only "accepted" rate.** Sample accepted output for real accuracy.

### What to say in an interview

"I treat a schema as eliminating syntax errors only. I'd make absent fields nullable to avoid fabrication, validate meaning in code, and retry only fixable errors, sending the document, the failed output and the specific error. Absent information goes to a person. Then I'd sample accepted output, since acceptance isn't accuracy." Add the current API detail if asked: forced `tool_choice` returns a 400 on the newest models, so I'd use structured outputs there.

## Certification lens

### How this lesson maps to CCAR-F

Direct: **TS 4.3**, **TS 4.4**, parts of **TS 2.3**, and Scenario 6. Remember: `auto` may return text, `any` must call some tool, and forced must call the named one. **The exam follows the guide, while the newest models reject forced values.**

These are original practice questions, not real or leaked exam questions.

**Q** Q1 · Exam style, scenario: Structured Data Extraction · Medium. Documents of unknown type must each produce structured output, and you have separate extraction tools for invoices, receipts and contracts. Sometimes the model replies in plain text. Which configuration addresses this, per the exam guide?

A. `tool_choice: "auto"` with a stronger instruction.

B. `tool_choice: "any"`, so that it must call a tool, while choosing which.

C. `tool_choice` forced to the invoice tool for every document.

D. Remove two of the tools, to reduce ambiguity.

**A** **B.** It guarantees a tool call when the document type is unknown (TS 4.3). A may still return text. C would run the wrong schema on receipts and contracts. D loses required capability. *Today's note: `any` returns a 400 on Opus 5.5, Sonnet 5.5 and Fable 5.1, where you'd use `auto` plus `strict` and a check, or structured outputs.*

**Q** Q2 · Exam style, scenario: Structured Data Extraction · Medium. A validation-retry loop runs three attempts on a form whose required tax ID appears only in a separate document you didn't provide. All three fail. What should you change?

A. Raise the retry limit to five.

B. Add more emphatic instructions on each retry.

C. Detect that the information is absent, stop retrying, and route to review or fetch the other document.

D. Mark the field as required in a stricter schema.

**A** **C.** Retries can't produce absent information (TS 4.4). A and B waste money and increase the risk of fabrication. D makes that pressure worse.

**Q** Q3 · Exam style, scenario: Structured Data Extraction · Hard. Choose the **two** schema changes that best reduce fabricated values and miscategorized records.

A. Make fields nullable when the source may lack them.

B. Add `"other"` with a detail string to the category enum.

C. Make every field required, so that nothing is skipped.

D. Remove the enum, and accept any string.

E. Raise `max_tokens`.

**A** **A and B** (TS 4.3). C causes the fabrication. D loses the consistency that downstream systems need. E has nothing to do with these errors.

**Q** Q4 · Configuration · Medium. A strict schema has `{"type": "number", "minimum": 0}`. What happens, and how do you enforce the rule?

**A** Numeric constraints aren't supported in strict schemas, so a raw schema gets a 400. SDK helpers may move the constraint into the field description, and check it during response validation. Either way, enforce "not negative" in your own code.

**Q** Q5 · Error identification · Hard. Find three problems: the retry prompt says "That was wrong, please try again." It runs up to 10 times. The schema has `purchase_order` as a required non-null string.

**A** (1) The feedback isn't specific. It needs the document, the failed output and the error. (2) Ten attempts wastes money, and repeating the pressure encourages invention. Stop after a small number, or immediately for absent data. (3) A required non-null field forces invention when the invoice has no PO. Make it nullable.

## Revision - Lesson 3

→ Schemas eliminate syntax errors, not semantic ones.

→ Required fields pressure the model to fabricate. Use nullable fields where the source may lack a value.

→ Enums need "other" + detail, and "unclear".

→ tool_choice: auto may return text, any must call some tool, forced must call a named tool.

→ Exam: tool use is the reliable route. Today: forced values return 400 on Opus 5.5, Sonnet 5.5 and Fable 5.1, so use output_config.format, or auto with strict.

→ Strict schemas reject numeric and string limits. Enforce those in code.

→ Validate meaning in code: totals, ranges and cross-field rules.

→ Retry with the document, the failed output and the specific error.

→ Retries fix format and structure. They can't find absent data.

→ calculated_total vs stated_total, and conflict_detected, flag contradictions.

→ detected_pattern enables analysis of why findings get dismissed.

→ Check stop_reason even with structured output.

Your notes